

LUMRA ERP

Page Architecture Blueprint

BAGIAN 3: PRODUCTION, ACCOUNTING & REPORTS

Disusun: 05 May 2026

DAFTAR ISI - BAGIAN 3

1. PRODUCTION MANAGEMENT

- Production Order List
- Create Production Order
- Production Order Detail & Execution
- Material Consumption Tracking
- Finished Goods Receipt

2. ACCOUNTING & GENERAL LEDGER

- Chart of Accounts
- General Ledger Entries
- Journal Entry Form
- Account Reconciliation

3. FINANCIAL REPORTS

- Balance Sheet
- Profit & Loss Statement
- Cash Flow Statement
- Trial Balance

4. OPERATIONAL REPORTS

- Sales Report (Daily/Monthly)
- Inventory Report
- Purchase Report
- Employee Performance

5. CUSTOM REPORT BUILDER

6. DASHBOARD & ANALYTICS

7. INTEGRATION & API BEST PRACTICES

8. LAMPIRAN FINAL

1. PRODUCTION MANAGEMENT

1.1 Production Order List

File Path: production/production_orders_list.html

UI/UX: Tabel: PO No, Output Product, Qty, Status (Draft, Confirmed, In Progress, Completed, Cancelled), Scheduled Date, Actual Date, Assigned To. Filter: Status, Date Range, Product. Search: PO No atau Product. Action: View, Edit (if Draft), Execute, Complete, Cancel

Backend: GET: Fetch ProductionOrders + prefetch product, user. Calculate completion % based on FinishedGoodsReceipts

Database: ProductionOrders (main), ProductionOrderLines (FK), FinishedGoodsReceipts (for qty completed tracking)

Flow: Production -> Orders -> List -> Click PO -> View / Edit / Execute / Complete

Status Colors: Draft=Gray, Confirmed=Blue, In Progress=Orange, Completed=Green, Cancelled=Red

1.2 Create Production Order

File Path: production/production_order_form.html

UI/UX: Tabs: Order (PO No auto-generate, Output Product dropdown, Target Qty, Scheduled Date, Notes), Ingredients (Tabel dari Recipe: Product, Required Qty, Unit, Available Qty with warning if low). Assign: Assign to operator

Backend: GET (create): Output product dropdown, recipe list. GET (edit): Pre-fill PO data. POST: Validate recipe exists for product, qty > 0. Create ProductionOrders + fetch recipe ingredients -> create ProductionOrderLines

Auto-Calculate: When output product selected, auto-load recipe from Recipes table. Show required materials

Database: ProductionOrders (write), ProductionOrderLines (write from recipe), Recipes (read for ingredients)

Flow: Click 'New PO' -> Select Output Product (auto-load recipe) -> Enter Qty & Date -> Auto-populate ingredients -> Assign to operator -> Save -> Show PO Detail

1.3 Production Order Detail & Execution

File Path: production/production_order_detail.html

UI/UX: Display: PO header (No, Product, Target Qty, Status, Assigned To). Tab 1 (Materials): Tabel ingredients dengan Required Qty, Used Qty (updated from MaterialConsumption), Balance, Status (On Hand, Shortage). Tab 2 (Progress): Steps (Start Production, Consume Materials, Finish & Receipt). Tab 3 (History): Timeline of all actions

Actions Buttons: 'Start Production' (change status to In Progress), 'Finish & Receipt' (record finished goods)

Material Consumption: Operator can manually log consumed materials via Material Consumption Tracking form

Finished Goods Receipt: Click 'Finish & Receipt' -> Enter received qty -> Create FinishedGoodsReceipts record -> Increase inventory -> Status = Completed

Database: ProductionOrders (read/write status), ProductionOrderLines (read), MaterialConsumption (read), FinishedGoodsReceipts (write)

Flow: PO Detail -> Start -> Log Material Consumption -> Click Finish -> Record Receipt Qty -> Complete PO -> Update Inventory

1.4 Material Consumption Tracking

File Path: production/material_consumption_form.html

UI/UX: Modal atau separate page. Show PO & materials. Tabel: Material, Required Qty, Previously Consumed, Qty to Consume (input field), Balance. Submit button creates material consumption record

Backend: GET: Show PO materials + sum previous consumption. POST: Create MaterialConsumption record (po_id, material_id, qty_consumed, timestamp). Validate: consumed qty <= required qty. Update balance display in real-time (AJAX)

Database: MaterialConsumption (write), ProductionOrderLines (read: required_qty)

Error Handling: If consumed qty > required qty -> show warning but allow (for overage tracking)

Flow: PO Detail -> Tab 'Materials' -> Click 'Log Consumption' -> Form -> Enter qty -> Submit -> Update material balance in real-time

1.5 Finished Goods Receipt

File Path: production/finished_goods_receipt_form.html

UI/UX: Modal. Show PO info + output product. Input: Received Qty, Received Date, Quality Status (Pass/Fail/Partial). Notes field for defects or issues

Backend: POST: Validate received_qty > 0 and <= target_qty (or allow overage?). Create FinishedGoodsReceipts record. Create InventoryTransaction to increase inventory. If received_qty < target_qty -> create variance record for investigation. Update ProductionOrders.status to COMPLETED

Database: FinishedGoodsReceipts (write), InventoryTransaction (write), ProductionOrders (update status)

Flow: PO Detail -> Finish & Receipt -> Modal -> Enter received qty & quality -> Submit -> Create receipt & update inventory -> PO Complete

2. ACCOUNTING & GENERAL LEDGER

2.1 Chart of Accounts

File Path: accounting/chart_of_accounts.html

UI/UX: Tree view hierarchical accounts (Asset, Liability, Equity, Income, Expense). Tabel columns: Account Code, Account Name, Type (Debit/Credit), Balance, Status. Action: View, Edit, Activate/Deactivate, View Ledger

Backend: GET: Fetch ChartOfAccounts dengan parent-child structure. Calculate balance per account dari GeneralLedger (SUM debit - credit)

Database: ChartOfAccounts (main with parent_id FK), GeneralLedger (for balance calculation)

Flow: Accounting -> Chart of Accounts -> Tree View -> Click account -> View Ledger / Edit / Deactivate

2.2 General Ledger Entries

File Path: accounting/general_ledger_list.html

UI/UX: Tabel: Date, Journal (manual entry / auto-generated from sales/purchase), Description, Debit Amount, Credit Amount, Reference No, Post Status (Draft/Posted). Filter: Date Range, Journal Type, Account. Search: Description atau Reference. Action: View, Edit (if Draft), Post, Reverse

Backend: GET: Fetch GeneralLedger entries. Show only posted entries (unless user is admin). Pagination with 50 entries per page

Database: GeneralLedger (main), ChartOfAccounts (FK)

Flow: Accounting -> General Ledger -> List -> Filter -> View Entry / Post / Reverse

2.3 Journal Entry Form (Manual)

File Path: accounting/journal_entry_form.html

UI/UX: Form: Entry Date, Journal Name (dropdown), Description, Line Items (tabel: Account dropdown, Debit, Credit, Description). Add/remove line buttons. Auto-validate: Total Debit = Total Credit

Backend: GET: Load chart of accounts for dropdown. POST: Validate total debit = total credit, all accounts exist, at least 2 lines. Create JournalEntries record + GeneralLedger entries (auto-generated)

Database: JournalEntries (write), GeneralLedger (write), ChartOfAccounts (read for dropdown)

Special: Status initially Draft. Admin posts entries for final approval

Flow: Accounting -> Journal Entries -> New -> Form -> Add lines -> Submit -> Validate balance -> Create entry (Draft) -> Post (admin only)

2.4 Account Reconciliation

File Path: accounting/account_reconciliation.html

UI/UX: Form: Select Account, Start Date, End Date, Bank Statement Balance (manual input), System Balance (auto-calculated). Reconciling items table: Outstanding Checks, Deposits in Transit, Errors. Difference amount with alert if not zero

Backend: GET: Select account -> show system balance (from GeneralLedger). Calculate: System Balance + Reconciling Items = Bank Statement Balance

Database: GeneralLedger (for system balance), Reconciliation (store reconciliation record)

Flow: Accounting -> Reconciliation -> Select Account & Date -> Compare with bank statement -> Add reconciling items -> Validate -> Post reconciliation

3. FINANCIAL REPORTS

3.1 Balance Sheet

File Path: reports/balance_sheet.html

UI/UX: Form: Report Date (datepicker). Display: 3 main sections (Assets, Liabilities, Equity) dengan sub-accounts indented. Tabel columns: Account, Amount. Totals & subtotals bolded. Export button (PDF, Excel)

Backend: GET: Report Date parameter -> fetch all accounts type Asset, Liability, Equity -> sum balances per account dari GeneralLedger. Calculate totals. Validate: Assets = Liabilities + Equity

Database: GeneralLedger (aggregate), ChartOfAccounts (structure)

PDF Export: Use WeasyPrint untuk render balance sheet in PDF format dengan company header

Flow: Reports -> Balance Sheet -> Select Date -> View Report -> Export PDF/Excel

3.2 Profit & Loss Statement

File Path: reports/profit_loss_statement.html

UI/UX: Form: Date Range (from-to datepickers). Display: Income - Expenses = Net Profit. Sections: Revenue (all income accounts), Expenses (all expense accounts), Operating Income, Other Income/Expense

Backend: GET: Date range -> fetch Income & Expense accounts -> sum amounts per account. Calculate: Total Revenue - Total Expenses = Net Profit

Database: GeneralLedger (aggregate by date range)

Flow: Reports -> P&L; -> Select Date Range -> View -> Export

3.3 Cash Flow Statement

File Path: reports/cash_flow_statement.html

UI/UX: Form: Date Range. Display: 3 sections (Operating Activities, Investing Activities, Financing Activities) + Beginning & Ending Cash Balance

Backend: GET: Complex calculation from GeneralLedger. Require manual mapping of accounts to activity categories (setup in admin)

Database: GeneralLedger, AccountActivityMapping (admin config)

Note: This requires predefined account mappings to activity types. Setup in admin before report can work

Flow: Reports -> Cash Flow -> Select Date Range -> View (requires account mapping setup) -> Export

3.4 Trial Balance

File Path: reports/trial_balance.html

UI/UX: Form: Report Date. Tabel: Account Code, Account Name, Debit Balance, Credit Balance. Totals row showing total debit & credit

Backend: GET: Report Date -> fetch all active accounts -> calculate debit & credit per account. Validate: Total Debit = Total Credit

Database: GeneralLedger (aggregate), ChartOfAccounts (list)

Alert: If total debit != total credit -> show error. Likely indicates posting errors

Flow: Reports -> Trial Balance -> Select Date -> View -> Export

4. OPERATIONAL REPORTS

4.1 Sales Report (Daily/Monthly)

File Path: reports/sales_report.html

UI/UX: Form: Date Range, Group By (Day/Week/Month), Filter by Product Category, Sales Channel (POS/SO). Display: Tabel atau chart. Columns: Date, Product Category, Qty Sold, Revenue, Avg Price. Summary: Total Revenue, Total Qty, Avg Transaction Value

Backend: GET: Date range + filters -> aggregate dari OrderLines. Group by date & category. Calculate: SUM(qty), SUM(revenue), AVG(price)

Database: Orders (filter by date), OrderLines (details), ProductItems (for category)

Chart: Line chart for revenue trend. Bar chart for product category comparison

Flow: Reports -> Sales -> Select Date Range & Filters -> View Report -> Export CSV/PDF

4.2 Inventory Report

File Path: reports/inventory_report.html

UI/UX: Form: Report Date, Location filter, Category filter. Display: Tabel: SKU, Product, Current Qty, Min Level, Max Level, Reorder Qty, Turnover Rate, Stock Value (qty * cost). Highlight: Low stock (yellow), Out of stock (red), Slow movers (blue)

Backend: GET: All products -> calculate current qty (from InventoryTransaction), turnover (sold last 30 days / avg qty). Show slow movers & overstock

Database: ProductItems, InventoryTransaction (for qty & turnover), OrderLines (for sold qty)

Flow: Reports -> Inventory -> Select filters -> View Report -> Identify issues -> Export

4.3 Purchase Report

File Path: reports/purchase_report.html

UI/UX: Form: Date Range, Vendor filter. Display: Tabel: PO No, Vendor, Product, Qty, Total Amount, Status. Summary: Total Purchase Amount, Total Qty, Top Vendors

Backend: GET: Date range -> aggregate dari PurchaseOrders & PurchaseOrderLines

Database: PurchaseOrders, PurchaseOrderLines, LumraConfigVendors

Flow: Reports -> Purchase -> Select filters -> View Report -> Export

5. CUSTOM REPORT BUILDER

5.1 Query Builder Interface

File Path: reports/custom_report_builder.html

UI/UX: Drag-drop interface untuk build custom report. Step 1: Select Table (Orders, Products, etc). Step 2: Select Columns to display. Step 3: Add Filters (conditions). Step 4: Grouping & Aggregation (SUM, COUNT, AVG). Step 5: Preview & Export

Backend: Build Django QuerySet dynamically based on user selections. Validate selections for security (prevent SQL injection)

Database: Any model defined in project can be queried

Special: Save custom reports as template for future use. Schedule report generation (daily/weekly email)

Flow: Reports -> Custom Report -> Select Table -> Columns -> Filters -> Preview -> Save Template -> Export

6. DASHBOARD & ANALYTICS

6.1 Main Executive Dashboard

File Path: dashboard/main_dashboard.html

UI/UX: KPI cards: Today Sales, Monthly Revenue, Active Users, Inventory Value, Pending POs. Charts: Revenue trend (line), Sales by category (pie), Top products (bar). Quick stats: Today transactions, Yesterday comparison, Month-to-date

Backend: Aggregate dari Orders, ProductItems, Users. Use cache untuk performa (5 min TTL). Prefetch metrics saat user login

Database: Orders (sales metrics), ProductItems (inventory), Users (active count)

Real-time: WebSocket untuk live update POS sales (optional, nice-to-have)

Flow: User login -> Dashboard loads -> Show KPIs & trends -> Drill-down to detailed reports

6.2 Department-Specific Dashboards

Sales Manager Dashboard: Top customers, sales rep performance, revenue vs target, pending orders

Inventory Manager Dashboard: Low stock alert, overstock, slow movers, stock opname status

Accounting Manager Dashboard: Outstanding AR, AP aging, trial balance status, GL posting status

Production Manager Dashboard: Production orders status, material consumption, finished goods received, yield variance

Implementation: Each dashboard GET from respective models, filter by department/user role

7. INTEGRATION & API BEST PRACTICES

7.1 REST API Standard Endpoints

- GET /api/products/ -> List products with pagination, filtering
- GET /api/products/[id]/ -> Product detail
- POST /api/products/ -> Create product
- PUT /api/products/[id]/ -> Update product
- DELETE /api/products/[id]/ -> Soft delete product
- GET /api/orders/ -> List orders
- POST /api/orders/ -> Create order (POS, SO)
- GET /api/orders/[id]/detail/ -> Order detail with lines
- POST /api/inventory/stock-adjustment/ -> Create stock adjustment
- GET /api/reports/sales-report/?date_from=X&date_to=Y -> Sales report (parametric)
- GET /api/dashboard/kpis/ -> Dashboard metrics

All endpoints require JWT token authentication (Authorization: Bearer TOKEN)

Throttling: 1000 requests/hour per user

Response format: {"success": bool, "data": {}, "message": string, "errors": {}}

7.2 Serializers & Validation

Use DRF ModelSerializer untuk auto-generate validators dari model fields. Custom validators untuk business logic (e.g., code uniqueness, stock availability). Nested serializers untuk related objects (Order + OrderLines in single endpoint). Pagination: LimitOffsetPagination dengan max_limit=100. Filtering: DjangoFilterBackend + SearchFilter untuk full-text search

7.3 Async Tasks (Celery)

Heavy operations di background tasks:

- Generate report PDF (> 100 pages) -> Task, email link
- Bulk import products (CSV) -> Task, notify on completion
- Send invoice email -> Task queue
- Calculate loyalty stamps -> Task (scheduled, hourly)
- Stock level alerts -> Periodic task (every hour)

Use Celery Beat untuk scheduled tasks. Store task results di Redis untuk quick retrieval

8. LAMPIRAN FINAL

8.1 Database Indexing Strategy

Create indexes on frequently queried fields:

- ProductItems: code (unique), name (for search)
- Orders: customer_id, order_date, status
- InventoryTransaction: product_id, location_id, transaction_date
- GeneralLedger: account_id, posting_date, journal_id
- StockOpnameSessions: location_id, created_at, status

Use compound indexes untuk common filter combinations

Monitor slow queries: Enable Django Debug Toolbar in dev, use Silk profiler in production

8.2 Security Best Practices

- CSRF Protection: Always include {% csrf_token %} in forms
- Permission Checks: @permission_required decorator on every sensitive view
- Rate Limiting: Throttle API endpoints to prevent abuse
- Input Validation: Sanitize & validate ALL user inputs (forms & API)
- SQL Injection Prevention: Use Django ORM QuerySet, never raw SQL unless absolutely necessary
- Soft Delete: Never show deleted data unless explicitly requested by admin
- Audit Logging: Log all create/update/delete operations with user & timestamp
- Two-Factor Auth: Implement 2FA for admin users
- HTTPS Only: Force HTTPS in production (DEBUG=False, SECURE_SSL_REDIRECT=True)

8.3 Testing Strategy

Unit Tests: Test individual models & view functions

Integration Tests: Test workflows (e.g., POS full transaction flow)

API Tests: Test all API endpoints with valid & invalid inputs

Load Tests: Test high-concurrency scenarios (many POS terminals at same time)

Fixtures: Create test data for repeatable test runs

Coverage: Aim for >80% code coverage on critical modules

CI/CD: Run tests automatically on every commit (GitHub Actions / Jenkins)

Example: @pytest.mark.django_db class TestPOSCheckout: def test_create_order_with_valid_items(): ...

8.4 Deployment Checklist

Pre-Deployment:

- All tests passing (coverage > 80%)
- Code review completed
- Database migrations reviewed
- Static files optimized (minified CSS/JS)
- Secrets (API keys, DB password) in environment variables
- Logging configured (Sentry for error tracking)

Post-Deployment:

- Health check monitoring (uptime, response time)
- Error tracking active (Sentry alerts)
- Database backups scheduled
- Cache warmed up
- Performance monitoring (slow queries, N+1 queries)
- User support ready for issues

8.5 Project Statistics

Total Halaman (Pages): ~60 halaman web + dashboard + reports

Total Models: ~50 Django models (dari rekap)

Total Views: ~80 view functions (list, form, detail, API)

Total API Endpoints: ~30 REST endpoints

Total Templates: ~100+ HTML templates

Estimated LOC (Lines of Code): ~20,000 LOC (Python + HTML + JS)

Database Tables: ~200+ tables

Average Load Time: < 2 seconds per page (with caching)

Concurrent Users Supported: 100-500+ with load balancing

8.6 Next Steps for Implementation

1. Start dengan Master Data & Settings (Bagian 1) - est. 2-3 weeks
2. Implement POS & Sales (Bagian 2) - est. 3-4 weeks
3. Add Production & Accounting (Bagian 3) - est. 3-4 weeks
4. Create Reports & Analytics - est. 2-3 weeks
5. Security hardening & penetration testing - est. 1-2 weeks
6. Performance tuning & load testing - est. 1-2 weeks
7. User training & documentation - est. 2 weeks
8. Go-live & support - est. ongoing

Total Estimated Timeline: 16-21 weeks (~4-5 months) dengan team 3-5 developers